

TypeScript

Le typage statique au service de JavaScript

TypeScript	Types	Interfaces	Génériques
------------	-------	------------	------------

tsconfig	tsc
----------	-----

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le fonctionnement du compilateur TypeScript et son rôle dans le processus de build
- ✓ Maîtriser le système de types : primitifs, unions, intersections, littéraux
- ✓ Concevoir des interfaces et types pour modéliser les données d'une application
- ✓ Utiliser les types génériques pour écrire du code réutilisable et type-safe
- ✓ Appliquer les Utility Types pour transformer des types existants
- ✓ Configurer tsconfig.json et intégrer TypeScript dans un projet Node.js et React

Table des matières

Chapitre 1 — Pourquoi TypeScript existe

- 1.1 Les limites de JavaScript dynamique
- 1.2 Le compilateur tsc
- 1.3 TypeScript dans l'écosystème

Chapitre 2 — Le système de types fondamental

- 2.1 Types primitifs et spéciaux
- 2.2 Union et intersection
- 2.3 Literal types et type narrowing
- 2.4 Type assertions

Chapitre 3 — Interfaces et types personnalisés

- 3.1 interface vs type
- 3.2 Propriétés optionnelles et readonly
- 3.3 Héritage et composition
- 3.4 Modéliser des APIs REST

Chapitre 4 — Génériques — écrire du code paramétrique

- 4.1 Fonctions génériques
- 4.2 Interfaces et classes génériques
- 4.3 Contraintes sur les génériques
- 4.4 Utility Types standards

Chapitre 5 — TypeScript en pratique

- 5.1 Configuration tsconfig.json
- 5.2 TypeScript avec React (JSX)
- 5.3 TypeScript avec Node.js/Express
- 5.4 Stratégies de migration

Pourquoi TypeScript existe

1.1 Les limites de JavaScript dynamique

JavaScript est un langage à typage dynamique : le type d'une variable est déterminé à l'exécution et peut changer à tout moment. Cette flexibilité est appréciable pour les petits scripts mais devient une source de bugs difficiles à détecter dans les applications de grande taille.

Considérons un exemple concret. Dans une application de commerce, une fonction `calculateTotal` reçoit un objet `order`. Si un développeur passe accidentellement une chaîne de caractères au lieu d'un nombre pour le prix, JavaScript fera de la coercition de type silencieuse — concaténant des chaînes au lieu d'additionner des nombres. Le bug n'apparaîtra qu'à l'exécution, peut-être des heures après le déploiement, sur des données de production.

Ambiguïté de typage en JavaScript

```
// JavaScript – aucune protection contre les erreurs de type

function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price * item.qty, 0);
}

// Ces appels sont tous valides pour JavaScript
calculateTotal([{ price: 100, qty: 2 }]); // ✓ 200
calculateTotal([{ price: '100', qty: 2 }]); // Retourne '1001001002' !
calculateTotal([{ prix: 100, quantite: 2 }]); // Retourne 0 silencieusement
calculateTotal(null); // Crash à l'exécution
```

TypeScript résout ces problèmes en ajoutant un système de types statique à JavaScript. Le compilateur `tsc` analyse le code avant l'exécution et signale toute incohérence de types. Dans l'exemple précédent, `tsc` aurait signalé une erreur dès que le développeur écrit `'100'` (string) là où un `number` est attendu.

1.2 Le compilateur tsc

TypeScript est un sur-ensemble de JavaScript : tout code JavaScript valide est du TypeScript valide. Le compilateur `tsc` transforme le code TypeScript en JavaScript standard — le navigateur et Node.js ne voient jamais de TypeScript, seulement du JavaScript. TypeScript est donc un outil de développement, pas un runtime.

Le processus de compilation comporte trois phases distinctes. D'abord le parsing : le code source est analysé et transformé en AST (Abstract Syntax Tree). Ensuite la vérification de types : TypeScript

inspecte l'AST, résout les types, et signale les erreurs. Enfin l'émission : le code TypeScript est transpilé vers JavaScript selon les paramètres de tsconfig.json.

```
# Installer TypeScript  
npm install -D typescript @types/node  
  
# Vérifier sans compiler (le plus utilisé en dev)  
npx tsc --noEmit  
  
# Compiler vers dist/  
npx tsc  
  
# Mode watch – recompile à chaque modification  
npx tsc --watch  
  
# Initialiser tsconfig.json  
npx tsc --init
```

1.3 TypeScript dans l'écosystème

TypeScript a été créé par Microsoft en 2012 et open-sourcé la même année. Depuis 2020, il est systématiquement dans le top 5 des langages les plus utilisés dans les sondages Stack Overflow. Angular l'utilise par défaut depuis sa refonte en 2016. React, Vue, Svelte et Next.js offrent tous un support TypeScript de première classe. Côté backend, NestJS est construit entièrement en TypeScript, et Express dispose d'excellentes définitions de types.

Le système de types fondamental

2.1 Types primitifs et spéciaux

TypeScript étend les types primitifs JavaScript avec des types supplémentaires pour couvrir tous les cas de figure. Comprendre la distinction entre `null`, `undefined`, `void`, `never` et `unknown` est essentiel pour écrire du TypeScript correct.

Panorama des types primitifs et spéciaux

```
// Types primitifs JavaScript
const name: string = 'Mohamed';
const age: number = 28;
const isActive: boolean = true;
const sym: symbol = Symbol('id');
const bigN: bigint = 9007199254740993n;

// Types spéciaux TypeScript
let a: null = null; // La valeur null, et rien d'autre
let b: undefined = undefined; // La valeur undefined, et rien d'autre

function log(msg: string): void { // Ne retourne rien (ou undefined implicite)
  console.log(msg);
}

function fail(msg: string): never { // Ne retourne JAMAIS (throw ou boucle infinie)
  throw new Error(msg);
}

let input: unknown = getUserInput(); // Valeur inconnue – doit être vérifiée avant usage
let anyVal: any = 'anything'; // Désactive le typage – à éviter
```

Définition — Le type `any`

`any` désactive complètement la vérification de types pour une valeur. C'est l'équivalent de revenir à JavaScript brut. Évitez-le — préférez `unknown` pour les valeurs dont vous ne connaissez pas le type a priori. La différence : `unknown` vous force à vérifier le type avant de l'utiliser ; `any` vous laisse faire n'importe quoi sans vérification.

2.2 Union et intersection

Les types union (A | B) et intersection (A & B) sont deux opérateurs fondamentaux pour composer des types existants. Un type union représente une valeur qui peut être de l'un OU l'autre type. Un type intersection représente une valeur qui est simultanément des deux types.

```
// Union – l'un OU l'autre
type StringOrNumber = string | number;
type Platform = 'whatsapp' | 'facebook' | 'tiktok';
type Nullable<T> = T | null;

// Utilisation pratique dans une API
function formatId(id: string | number): string {
  return typeof id === 'number' ? id.toString() : id;
}

// Intersection – les DEUX à la fois
type Timestamped = { created_at: string; updated_at: string };
type TenantScoped = { tenant_id: string };

type AuditableEntity = Timestamped & TenantScoped;
// = { created_at, updated_at, tenant_id } – toutes les propriétés

// Très utile pour enrichir des types existants
type ProductWithStock = Product & { stock_quantity: number };
```

2.3 Literal types et type narrowing

TypeScript permet de définir des types qui n'acceptent que des valeurs spécifiques — pas juste 'une string' mais 'exactement cette string'. C'est extrêmement puissant pour modéliser des états finis, comme les statuts d'une commande.

Le type narrowing est le mécanisme par lequel TypeScript réduit le type d'une variable dans un bloc conditionnel. Quand vous écrivez `if (typeof x === 'string')`, TypeScript sait que dans ce bloc, `x` est forcément une string. Ces 'type guards' peuvent utiliser `typeof`, `instanceof`, des discriminated unions, ou des fonctions personnalisées.

```
// Literal types – valeurs exactes
type OrderStatus = 'new' | 'confirmed' | 'processing' | 'delivered' | 'cancelled';
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE';

// Discriminated union – pattern très puissant
type ApiResponse<T> =
  | { success: true; data: T }
  | { success: false; error: string };
```



```
function handleResponse<T>(res: ApiResponse<T>): T {
  if (res.success) {
    return res.data; // TypeScript sait que res.data existe ici
  } else {
    throw new Error(res.error); // et que res.error existe ici
  }
}

// Type guards personnalisés
function isString(val: unknown): val is string {
  return typeof val === 'string';
}
```

Interfaces et types personnalisés

3.1 interface vs type

TypeScript offre deux façons de nommer et réutiliser des types : interface et type alias. Ils se ressemblent beaucoup mais ont des différences importantes. Une interface décrit la forme d'un objet et peut être étendue (extends) ou fusionnée (declaration merging). Un alias type peut décrire n'importe quel type — pas seulement des objets — mais ne supporte pas la declaration merging.

```
// interface – pour les objets et les classes
interface User {
  id: string;
  email: string;
  role: 'merchant' | 'agent' | 'admin';
}

// Peut être étendu
interface AdminUser extends User {
  permissions: string[];
}

// type – pour les unions, intersections, alias
type ID = string;
type Platform = 'whatsapp' | 'facebook' | 'tiktok';
type UserOrAdmin = User | AdminUser;

// Règle pratique :
// interface → forme d'un objet ou d'une classe
// type → tout le reste (unions, intersections, alias simples)
```

3.2 Propriétés optionnelles et readonly

Le modificateur ? rend une propriété optionnelle — elle peut être absente. readonly interdit la réassignation après initialisation. Ces deux outils permettent de modéliser précisément les contrats de données.

```
interface CreateProductBody {
  name: string; // obligatoire
```

```

price: number; // obligatoire
description?: string; // optionnel
stock_quantity?: number; // optionnel (défaut côté serveur)
}

interface ProductFromDB {
  readonly id: string; // ne peut pas être réassigné
  readonly tenant_id: string; // ne peut pas être réassigné
  name: string;
  price: number;
  created_at: string;
  deleted_at: string | null; // soft delete
}

const product: ProductFromDB = { ... };
product.name = 'Nouveau nom'; // ✓ autorisé
product.id = 'autre-id'; // ■ Erreur TypeScript – readonly

```

3.3 Héritage et composition

TypeScript supporte l'héritage d'interfaces via `extends`, et la composition via les intersections. L'héritage est adapté quand un type est une spécialisation d'un autre ('un `AdminUser` EST un `User`'). La composition convient pour ajouter des capacités transversales ('un `Product` EST timestampé ET tenant-scopé').

3.4 Modéliser des APIs REST

Un des cas d'usage les plus importants de TypeScript est le typage des APIs. Définir des types pour les request bodies, les réponses, les paramètres de route et les query params transforme radicalement la fiabilité du code backend.

```

// Typer une route Express complètement
import { Request, Response } from 'express';

interface OrderParams {
  id: string;
}

interface PatchOrderBody {
  status: OrderStatus;
}

```

```
interface OrderResponse {
  id: string;
  status: OrderStatus;
  total_amount: number;
  items: OrderItem[];
}

// Route typée – erreurs détectées à la compilation
app.patch('/orders/:id',
  async (req: Request<OrderParams, OrderResponse, PatchOrderBody>, res) => {
    const { id } = req.params; // string – typé
    const { status } = req.body; // OrderStatus – typé
    // ...
  }
);
```

Génériques — écrire du code paramétrique

4.1 Fonctions génériques

Un type générique est un type paramétré : il accepte un ou plusieurs types en paramètre, comme une fonction accepte des arguments. Le symbole `T` (par convention, mais tout identifiant fonctionne) représente le type paramètre. Les génériques permettent d'écrire du code qui fonctionne avec n'importe quel type tout en préservant la sécurité du typage.

```
// Sans générique – on perd l'information de type
function first(arr: any[]): any {
  return arr[0];
}
const f = first([1, 2, 3]); // type : any – pas utile

// Avec générique – le type est préservé
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
const f1 = first([1, 2, 3]); // type : number | undefined
const f2 = first(['a', 'b', 'c']); // type : string | undefined

// TypeScript infère T depuis les arguments – pas besoin de l'écrire
const f3 = first<Order>(orders); // explicite
const f4 = first(orders); // inféré – même résultat
```

4.2 Contraintes sur les génériques

Par défaut, un générique `T` peut être n'importe quel type. La clause `extends` permet de contraindre `T` à être un sous-type d'un type donné. Cela permet d'accéder aux propriétés du type contrainte dans la fonction générique.

```
// Contrainte : T doit avoir une propriété 'id'
function findById<T extends { id: string }>(items: T[], id: string): T | undefined {
  return items.find(item => item.id === id);
}

// Fonctionne avec Order, Product, User, etc.
```

```

const order = findById(orders, '123'); // T inféré : Order
const product = findById(products, '456'); // T inféré : Product

// Contrainte avec keyof – accès type-safe aux propriétés
function getField<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const name = getField(product, 'name'); // type : string
const price = getField(product, 'price'); // type : number
// getField(product, 'invalid') // ■ Erreur TypeScript

```

4.3 Utility Types standards

TypeScript fournit une bibliothèque de types génériques prédéfinis appelés Utility Types. Ils permettent de transformer des types existants sans les réécrire. Ce sont des outils du quotidien dans tout projet TypeScript sérieux.

Utility Types les plus utilisés

Utility Type	Transformation	Exemple
Partial	Toutes les propriétés optionnelles	Partial — pour PATCH
Required	Toutes les propriétés obligatoires	Required
Readonly	Toutes les propriétés readonly	Readonly
Pick	Garder seulement certaines clés	Pick
Omit	Exclure certaines clés	Omit
Record	Objet avec clés K et valeurs V	Record
ReturnType	Type de retour d'une fonction	ReturnType

TypeScript en pratique

5.1 Configuration tsconfig.json

Le fichier `tsconfig.json` configure le compilateur TypeScript pour le projet. Les options sont nombreuses mais quelques-unes sont critiques pour un projet professionnel. Le mode strict active un ensemble d'options de vérification strictes qui éliminent les erreurs les plus courantes.

tsconfig.json recommandé pour un backend Node.js

```
{
  "compilerOptions": {
    // Cible et format de sortie
    "target": "ES2020", // Version JS de sortie
    "module": "commonjs", // Format des modules (Node.js)
    "lib": ["ES2020"], // APIs standard disponibles

    // Strictness – toujours activer en 2024
    "strict": true, // Active les 8 options strict* en une fois
    "noUncheckedIndexedAccess": true, // arr[0] est T | undefined
    "exactOptionalPropertyTypes": true,

    // Chemins
    "rootDir": "./src",
    "outDir": "./dist",
    "baseUrl": ".",

    // Interopérabilité
    "esModuleInterop": true, // import X from "x" fonctionne
    "resolveJsonModule": true, // import data from "./data.json"
    "skipLibCheck": true // Ignorer les erreurs dans node_modules
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "src/test/**"]
}
```

5.2 TypeScript avec React (JSX)

Dans un projet React avec TypeScript, les fichiers utilisent l'extension .tsx. La principale différence est le typage des props, du state et des événements. Expo/React Native supporte TypeScript nativement.

```
// Typer les props d'un composant React
interface ConversationCardProps {
  conversation: Conversation;
  onPress: (id: string) => void;
  isUnread?: boolean;
}

export function ConversationCard({
  conversation,
  onPress,
  isUnread = false,
}: ConversationCardProps) {
  return (
    <TouchableOpacity onPress={() => onPress(conversation.id)}>
    { /* ... */ }
    </TouchableOpacity>
  );
}

// Typer useState
const [orders, setOrders] = useState<Order[]>([]);
const [loading, setLoading] = useState<boolean>(false);

// Typer les événements
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setName(e.target.value);
};
```

■ À retenir

- ✓ TypeScript est un sur-ensemble de JS qui se compile — le navigateur/Node ne voit que du JS
- ✓ any désactive le typage — préférer unknown pour les valeurs de type inconnu
- ✓ Les discriminated unions permettent un narrowing précis et élégant
- ✓ interface pour les objets ; type pour les unions, intersections et alias
- ✓ Les génériques permettent d'écrire du code paramétrique type-safe sans dupliquer
- ✓ Toujours activer strict: true dans tsconfig.json

Exercice 1 — Modéliser le domaine

Créer un fichier `types/domain.ts` qui modélise toutes les entités de votre application.

- Définir les interfaces pour `User`, `Tenant`, `Product`, `Order`, `OrderItem`
- Utiliser des literal types pour tous les statuts et plateformes
- Ajouter `Timestamped` et `TenantScoped` comme types de composition
- Vérifier avec `tsc --noEmit` que tout est cohérent

Exercice 2 — Typer une API complète

Typer une route Express de A à Z sans aucun `any`.

- Définir les types pour les params, body et réponse
- Typer le handler complet : `Request`, `Response`
- Utiliser `Partial` pour un endpoint `PATCH`
- Ajouter un type guard pour valider le body entrant

Exercice 3 — Refactoring avec les Utility Types

Utiliser les Utility Types pour éviter la duplication de types.

- Créer `CreateOrderBody` depuis `Order` avec `Omit` et `Partial`
- Créer `UpdateOrderBody` avec `Pick` pour n'exposer que les champs modifiables
- Créer une fonction générique `paginate` qui retourne `{ data: T[], total: number }`
- Vérifier que l'autocomplétion fonctionne correctement dans VS Code

Bibliographie & Ressources

Documentation officielle

- typescriptlang.org/docs/handbook — le Handbook TypeScript (référence absolue)
- typescriptlang.org/tsconfig — documentation complète de tsconfig.json
- typescriptlang.org/play — bac à sable TypeScript en ligne

Formations recommandées

- Matt Pocock — Total TypeScript (totaltypescript.com) — formations interactives
- Execute Program — TypeScript (executeprogram.com) — apprentissage par la pratique

Articles de référence

- basarat.gitbook.io/typescript — TypeScript Deep Dive (gratuit)
- effectivetypescript.com — Dan Vanderkam, 62 ways to improve your TypeScript